

1. Business Understanding

Problem Statement

In the modern digital landscape, businesses need to understand how customers perceive their products. Twitter provides a vast stream of real-time customer opinions, but manually reviewing and analyzing these tweets is time-consuming and infeasible at scale. Businesses that fail to detect and address negative sentiment early may suffer reputational and financial damage.

Business Need

Automated sentiment classification helps businesses quickly assess public opinion. With such tools, companies can:

- Identify popular or problematic products.
- Monitor changes in sentiment over time.
- React proactively to criticism and improve product offerings.
- Enhance marketing and customer service strategies.

Project Objective

To develop a predictive model that classifies the sentiment of a tweet; **positive**, **negative**, or **neutral**, based solely on its raw text content.

Business Value

A robust sentiment analysis model enables:

- Real-time brand reputation monitoring.
- Strategic product decisions based on user feedback.
- Data-driven A/B testing and product improvement.

Project Requirements

- A labeled dataset of tweets

- Preprocessing pipeline to clean noisy social media text
- Computational resources for model training
- Evaluation framework for model performance
- A team of data scientists and domain knowledge experts

Challenges

- Noisy and informal language in tweets
- Ensuring class balance and clean input data
- Making predictions explainable to business stakeholders

Solutions

- Used a publicly available **CrowdFlower Twitter sentiment dataset**
 - Applied text preprocessing using **NLTK**, **regex**, and a custom **TweetPreprocessor** class
 - Developed multiple models including logistic regression and an ensemble stack
 - Applied **LIME** for model interpretability
-

2. Data Understanding

Dataset Overview

The dataset consists of tweets that reference major tech brands (e.g., Apple, Google). Each record includes:

- **tweet_text**: Raw tweet content
- **brand_reference**: Brand mentioned in the tweet

- `is_there_an_emotion_directed_at_a_brand_or_product`: Labeled sentiment/emotion

Target Sentiment Classes

- **Positive emotion**
- **Negative emotion**
- **No emotion toward brand or product**
- **I can't tell**

Observations

- Tweets were often short, informal, and full of noise (e.g., hashtags, emojis, URLs).
- Class imbalance existed but was manageable.
- A few tweets had missing text; these were removed.

Text Characteristics

- Frequent use of slang, emojis, mentions (@), and links.
- Repeated letters, misspellings, and abbreviations were common.
- Required thorough preprocessing to make suitable for modeling.

3. Data Preparation

Preprocessing Pipeline

We built a **custom class** called `TweetPreprocessor`, which:

- Inherited from `BaseEstimator` and `TransformerMixin` (from Scikit-learn)

- Could be easily plugged into ML pipelines for transformation

Cleaning Steps Included:

- Lowercasing text
- Removing emojis, URLs, hashtags, and mentions
- Filtering out punctuation and special characters
- Reducing repeated letters
- Tokenization and stopwords removal using **NLTK**
- Lemmatization for standardizing word forms
- Output formats: token list (`clean_tokens`) and cleaned string (`clean_text`)

Vectorization

- TF-IDF vectorizer was used to convert `clean_text` into numerical features for modeling.
-

4. Modeling

♦ Baseline Model: Logistic Regression

We implemented a baseline model using Scikit-learn's `LogisticRegression`, embedded in a pipeline with our custom `TweetProcessor` and a TF-IDF vectorizer.

To improve performance, we applied **GridSearchCV** for hyperparameter tuning. The search focused on both the text vectorization parameters (like `ngram_range`, `max_df`, and `min_df`) and model-specific parameters such as the regularization strength `C`.

Results:

- Accuracy before tuning: **56%**

- Accuracy after tuning: **64%**

This demonstrated that careful parameter tuning can significantly enhance model performance, even with a relatively simple algorithm.

♦ **Stacked Model: Logistic Regression, XGBoost, and SVC**

For improved performance and robustness, we implemented a **stacking ensemble** combining three diverse classifiers: `LogisticRegression`, `XGBoostClassifier`, and `SVC`, with a final `LogisticRegression` estimator.

The full pipeline included text preprocessing (`TweetProcessor` and `TfidfVectorizer`) followed by the stacking model. We used `GridSearchCV` to tune hyperparameters across all three base learners and the vectorizer, with a weighted F1-score as the evaluation metric.

Despite the extensive tuning, optimizing parameters like `C`, kernel types, `n_estimators`, and `max_depth`, there was **no observable improvement** in performance after tuning.

Results:

- Accuracy before tuning: **68%**
- Accuracy after tuning: **68%**

This outcome indicates that the stacking model reached its performance ceiling, likely due to data or feature constraints rather than model complexity.

SMOTE for Class Balancing

To address class imbalance, we experimented with **SMOTE** (Synthetic Minority Oversampling Technique). Although it successfully balanced the training data by synthetically generating minority class samples, the final model performance dropped:

- **Stacked model with SMOTE:** Accuracy decreased to **64%**

This suggests that synthetic examples may have introduced noise or distorted the true data distribution in this context.

5. Evaluation

Metrics Used:

- **Accuracy** (primary)
- **Classification Report:** Precision, Recall, F1-score

Key Findings:

- Ensemble stacking outperformed the logistic regression baseline.
 - Hyperparameter tuning significantly improved the baseline but not the stacked model.
 - Class-level evaluation showed that the model had more difficulty with ambiguous classes like “I can’t tell.”
-

6. Model Interpretability

LIME (Local Interpretable Model-Agnostic Explanations)

We applied **LIME** to interpret predictions and improve transparency:

- Used `LimeTextExplainer` from the `lime` package.
- Enabled row-specific explanation by using a dynamic `idx` parameter (e.g., `idx = 697`).
- LIME highlighted the top contributing words that influenced sentiment predictions.
- Visual outputs showed token contributions and confidence levels.

Benefits:

- Validated that the model focused on meaningful keywords.
 - Helped identify potential sources of error (e.g., sarcasm, ambiguous phrases).
 - Enhanced trustworthiness of model predictions for non-technical stakeholders.
-

7. Conclusion and Next Steps

Summary of Results

Model	Accuracy Before Tuning	Accuracy After Tuning
Logistic Regression	56%	64%
Stacked Classifier	68%	68%
Stacked Classifier + Undersampling majority classes	—	54%

Key Takeaways

- The stacked model yielded the best overall performance.
- Hyperparameter tuning benefited the baseline more than the ensemble.
- The data was very unbalanced, when we balanced the rows by undersampling the majority classes we were left with 18000 classes for training, which was not enough for the model to learn since we only got 54% accuracy.

Next Steps

- Explore deep learning models such as BERT or RoBERTa for context-aware predictions.

- Apply class weighting or stratified sampling as alternatives to SMOTE.
- Deploy the best-performing model using **FastAPI** or **Streamlit**.
- Add real-time Twitter scraping and dashboard analytics for end-users.